

Unicast Routing

Unit-III

Lecture - 4

August 28, 2019

Session Objectives

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- To Introduce the concept of unicast routing
- To discuss common routing algorithms used in the Internet
- To explore unicast-routing protocols:RIP, OSPF, BGP

Session Outcomes

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

At the end of this session, participants will be able to

- Discuss the unicast routing concept and routing algorithms

Agenda

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

1 Introduction

2 Routing Algorithms

- Distance-Vector Routing
- Link-State Routing
- Path-Vector Routing

Presentation Outline

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

1 Introduction

2 Routing Algorithms

- Distance-Vector Routing
- Link-State Routing
- Path-Vector Routing

Introduction

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- Unicast routing in the Internet, with a large number of routers and a huge number of hosts, can be done only by using hierarchical routing:.
- Routing in several steps using different routing algorithms.
- In unicast routing, a packet is routed, hop by hop, from its source to its destination by the help of forwarding tables.
- Routing a packet from its source to its destination means **routing the packet from a source router** (the default router of the source host) to a **destination router**
- Question is what other routers the packet should visit; which route the packet should take

Internet as a Graph

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

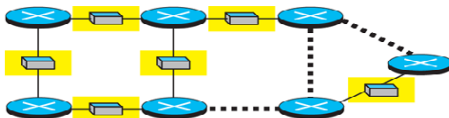
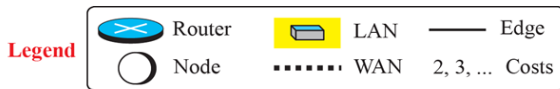
Distance-Vector

Routing

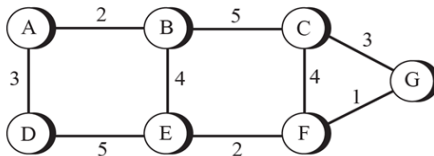
Link-State Routing

Path-Vector Routing

- Each router as a node and each network between a pair of routers as an edge; **internet can be modeled as a weighted graph** in which each edge is associated with a cost
- If there is no edge between the nodes, the cost is infinity.



a. An internet



b. The weighted graph

Least-Cost Routing

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector

Routing

Link-State Routing

Path-Vector Routing

- find the least cost between source router to the destination router
- N routers in an internet, then $(N - 1)$ least-cost paths from each router to any other router. we need $N \times (N - 1)$ least-cost paths for the whole internet
- **Least-Cost Trees:** a tree with the source router as the root that spans the whole graph (visits all other nodes) and in which the path between the root and any other node is the shortest
-

Unicast Routing

Unit-III

Introduction

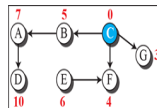
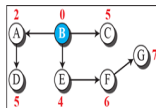
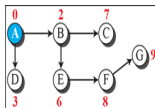
Routing Algorithms

Distance-Vector Routing

Routing

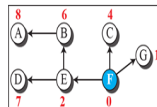
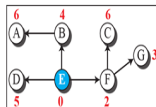
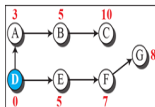
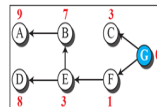
Link-State Routing

Path-Vector Routing



Legend

-  Root of the tree
-  Intermediate or end node
- 1, 2, ...** Total cost from the root



Least-cost trees properties

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector

Routing

Link-State Routing

Path-Vector Routing

- The least-cost route from X to Y in X's tree is the inverse of the least-cost route from Y to X in Y's tree;
- Instead of travelling from X to Z using X's tree, we can travel from X to Y using X's tree and continue from Y to Z using Y's tree

Presentation Outline

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

1 Introduction

2 Routing Algorithms

- Distance-Vector Routing
- Link-State Routing
- Path-Vector Routing

Distance-Vector(DV) Routing

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- Each node creates its own least-cost tree with the rudimentary information it has about its immediate neighbors.
- a router continuously tells all of its neighbors what it knows about the whole internet
- **Bellman-Ford Equation:** to find the least cost (shortest distance) between a source node, x , and a destination node, y , through some intermediary nodes ($a, b, c, \dots$)

$$D_{xy} = \min \{ (c_{xa} + D_{ay}), (c_{xb} + D_{by}), (c_{xc} + D_{cy}), \dots \}$$

- D_{ij} is the shortest distance and c_{ij} is the cost between nodes i and j .

Distance-Vector(DV) Routing

Unicast
Routing

Unit-III

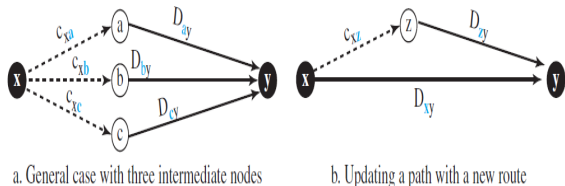
Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing



- Bellman-Ford equation enables us to build a new least-cost path from previously established least-cost paths
- we can think of $(a \rightarrow y)$, $(b \rightarrow y)$, and $(c \rightarrow y)$ as previously established least-cost paths and $(x \rightarrow y)$ as the new least-cost path.
- In distance-vector routing, normally we want to update an existing least cost with a least cost through an intermediary node, such as z

Distance-Vector(DV) Routing

Unicast
Routing

Unit-III

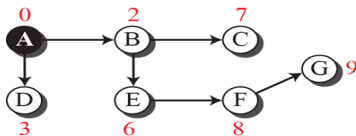
Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing



a. Tree for node A

	A
A	0
B	2
C	7
D	3
E	6
F	8
G	9

b. Distance vector for node A

- Name of the distance vector defines the root, the indexes define the destinations, and the value of each cell defines the least cost from the root to the destination.
- A distance vector **does not give the path** to the destinations as the least-cost tree does; it **gives only the least costs** to the destinations

First distance vector for an internet

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- The node sends some greeting messages out of its interfaces and discovers the **identity of the immediate neighbors and the distance between itself and each neighbor.**
- these vectors are made asynchronously
- After each node has created its vector, it sends a copy of the vector to all its immediate neighbors. A
- After a node receives a distance vector from a neighbor, it updates its distance vector using the Bellman-Ford equation (second case)

$$D_{xy} = \min \{ D_{xy}, (c_{xz} + D_{zy}) \}$$

Updating distance vectors

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

New B	Old B	A
A 2	A 2	A 0
B 0	B 0	B 2
C 5	C 5	C ∞
D 5	D ∞	D 3
E 4	E 4	E ∞
F ∞	F ∞	F ∞
G ∞	G ∞	G ∞

$B[] = \min(B[], 2 + A[])$

Note:

$X[]$: the whole vector

a. First event: B receives a copy of A's vector.

New B	Old B	E
A 2	A 2	A ∞
B 0	B 0	B 4
C 5	C 5	C ∞
D 5	D 5	D 5
E 4	E 4	E 0
F 6	F ∞	F 2
G ∞	G ∞	G ∞

$B[] = \min(B[], 4 + E[])$

Distance-Vector Routing Algorithm for A Node

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

```
1 Distance_Vector_Routing ( )
2 {
3     // Initialize (create initial vectors for the node)
4     D[myself] = 0
5     for (y = 1 to N)
6     {
7         if (y is a neighbor)
8             D[y] = c[myself][y]
9         else
10            D[y] = ∞
11    }
12    send vector {D[1], D[2], ..., D[N]} to all neighbors
13    // Update (improve the vector with the vector received from a neighbor)
14    repeat (forever)
15    {
16        wait (for a vector Dw from a neighbor w or any change in the link)
17        for (y = 1 to N)
18        {
19            D[y] = min [D[y], (c[myself][w] + Dw[y])]    // Bellman-Ford equation
20        }
21        if (any change in the vector)
22            send vector {D[1], D[2], ..., D[N]} to all neighbors
23    }
24 } // End of Distance Vector
```

Count to Infinity

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- Any **decrease in cost** (good news) propagates **quickly**, but any **increase in cost** (bad news) will propagate **slowly**.
- If a **link is broken** (cost becomes infinity), every other router should be aware of it immediately, but in **distance-vector routing, this takes some time**.
- It sometimes **takes several updates** before the cost for a broken link is recorded as infinity by all routers

Two-Node Loop

Unicast
Routing

Unit-III

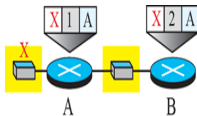
Introduction

Routing
Algorithms

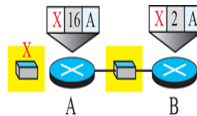
Distance-Vector
Routing

Link-State Routing

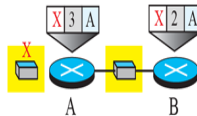
Path-Vector Routing



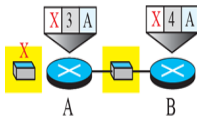
a. Before failure



b. After link failure

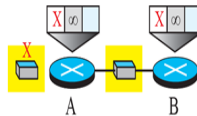


c. After A is updated by B



d. After B is updated by A

...



e. Finally

Split Horizon

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- Instead of flooding the table through each interface, each node sends only part of its table through each interface.
- Taking information from node A, modifying it, and sending it back to node A is what creates the confusion.
- In our scenario, node B eliminates the last line of its forwarding table before it sends it to A. In this case, node A keeps the value of infinity as the distance to X.
- Later, when node A sends its forwarding table to B, node B also corrects its forwarding table.
- The system becomes stable after the first update: both node A and node B know that X is not reachable

Poison Reverse

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector

Routing

Link-State Routing

Path-Vector Routing

- Split-horizon strategy has one drawback; the corresponding protocol uses a timer, and if there is no news about a route, the node deletes the route from its table.
- In the poison reverse strategy B can still advertise the value for X, but if the source of information is A, it can replace the distance with infinity as a warning: "Do not use this value; what I know about this route comes from you."

Link-State Routing

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- The **cost** associated with an edge defines the **state of the link**.
- Links with **lower costs are preferred** to links with higher costs;
- If the cost of a link is infinity, it means that the link does not exist or has been broken.
- Each node needs to have a complete map of the network, which means it needs to know the state of each link.
- The **collection of states for all links** is called the **link-state database (LSDB)**.
- There is only one LSDB for the whole internet; each node needs to have a duplicate of it

Link-State Routing

Unicast
Routing

Unit-III

Introduction

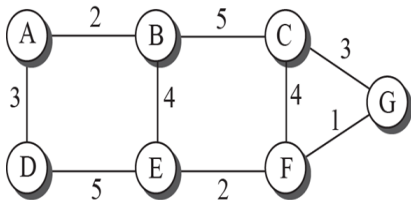
Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- The LSDB can be represented as a two-dimensional array(matrix) in which the value of each cell defines the cost of the corresponding link.

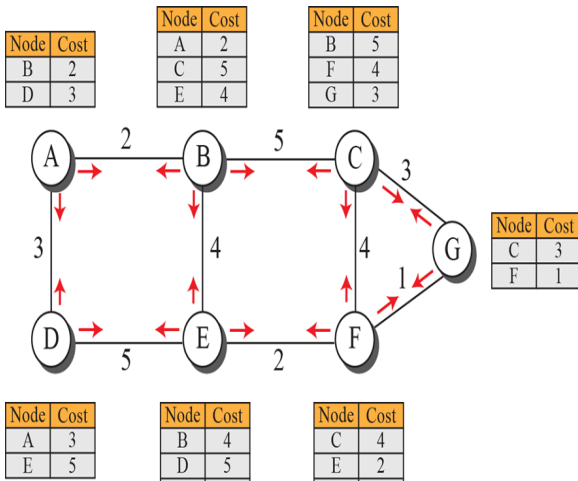


a. The weighted graph

	A	B	C	D	E	F	G
A	0	2	∞	3	∞	∞	∞
B	2	0	5	∞	4	∞	∞
C	∞	5	0	∞	∞	4	3
D	3	∞	∞	0	5	∞	∞
E	∞	4	∞	5	0	2	∞
F	∞	∞	4	∞	2	0	1
G	∞	∞	3	∞	∞	1	0

Flooding

- Link State info of neighbours are collected and tabulated
- LS Packets have sequence number that help to remove old info from their table



Formation of Least-Cost Trees: Dijkstra Algorithm

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- 1 The node chooses itself as the root of the tree, creating a tree with a single node, and sets the total cost of each node based on the information in the LSDB.
- 2 The node selects one node, among all nodes not in the tree, which is closest to the root, and adds this to the tree. After this node is added to the tree, the cost of all other nodes not in the tree needs to be updated because the paths may have been changed.
- 3 The node repeats step 2 until all nodes are added to the tree.

Dijkstra Algorithm

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector

Routing

Link-State Routing

Path-Vector Routing

```
1 Dijkstra's Algorithm ( )
2 {
3     // Initialization
4     Tree = {root}           // Tree is made only of the root
5     for (y = 1 to N)       // N is the number of nodes
6     {
7         if (y is the root)
8             D[y] = 0        // D[y] is shortest distance from root to node y
9         else if (y is a neighbor)
10            D[y] = c[root][y] // c[x][y] is cost between nodes x and y in LSDB
11        else
12            D[y] = ∞
13    }
14    // Calculation
15    repeat
16    {
17        find a node w, with D[w] minimum among all nodes not in the Tree
18        Tree = Tree ∪ {w}      // Add w to tree
19        // Update distances for all neighbor of w
20        for (every node x, which is neighbor of w and not in the Tree)
21        {
22            D[x] = min{D[x], (D[w] + c[w][x])}
23        }
24    } until (all nodes included in the Tree)
25 } // End of Dijkstra
```

Path-Vector Routing

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

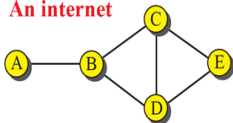
Distance-Vector
Routing

Link-State Routing

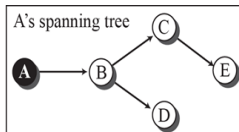
Path-Vector Routing

- When least-cost goal is not the priority; reach destination more efficiently without assigning costs to the route
- The source can control the path
- **Spanning Trees:** path from a source to all destinations is also determined by the best spanning tree.
- Each source has created its own spanning tree that meets its policy.
- The policy imposed by all sources is to use the minimum number of nodes to reach a destination.
- The spanning tree selected by A and E is such that the communication does not pass through D as a middle node.
- Similarly, the spanning tree selected by B is such that the communication does not pass through C as a middle node.

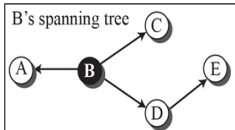
An internet



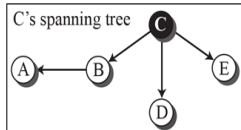
A's spanning tree



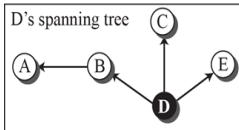
B's spanning tree



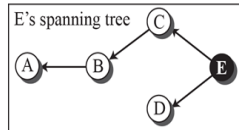
C's spanning tree



D's spanning tree



E's spanning tree



Creation of Spanning Trees

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- Path-vector routing, is an asynchronous and distributed routing algorithm.
- When a node is booted, it creates a path vector based on the information it can obtain about its immediate neighbor.
- A node sends greeting messages to its immediate neighbors to collect these pieces of information

Unicast Routing

Unit-III

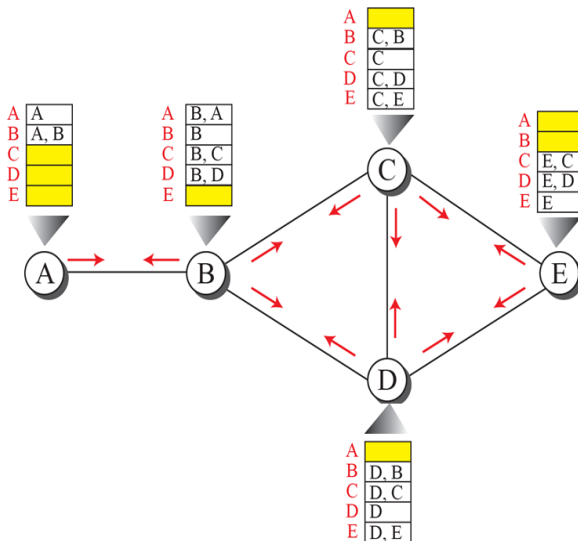
Introduction

Routing Algorithms

Distance-Vector Routing

Link-State Routing

Path-Vector Routing



Path vectors

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

- Each node, after the creation of the initial path vector, sends it to all its immediate neighbors.
- Each node, when it receives a path vector from a neighbor, updates its path vector using an equation similar to the Bellman-Ford, but applying its own policy instead of looking for the least cost.

$$\text{Path}(x, y) = \text{best} \{ \text{Path}(x, y), [(x + \text{Path}(\mathbf{v}, y))] \} \quad \text{for all } \mathbf{v}\text{'s in the internet.}$$

- If Path (v, y) includes x, that path is discarded to avoid a loop in the path

Creation of Spanning Trees

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

```
1 Path_Vector_Routing ( )
2 {
3     // Initialization
4     for (y = 1 to N)
5     {
6         if (y is myself)
7             Path[y] = myself
8         else if (y is a neighbor)
9             Path[y] = myself + neighbor node
10        else
11            Path[y] = empty
12    }
13    Send vector { Path[1], Path[2], ..., Path[y] } to all neighbors
14    // Update
15    repeat (forever)
16    {
17        wait (for a vector Pathw from a neighbor w)
18        for (y = 1 to N)
19        {
20            if (Pathw includes myself)
21                discard the path // Avoid any loop
22            else
23                Path[y] = best { Path[y], (myself + Pathw[y]) }
24        }
25        If (there is a change in the vector)
26            Send vector { Path[1], Path[2], ..., Path[y] } to all neighbors
27    }
28 } // End of Path Vector
```

Path-Vector Algorithm

Unicast
Routing

Unit-III

Introduction

Routing
Algorithms

Distance-Vector
Routing

Link-State Routing

Path-Vector Routing

	New C	Old C	B
A	C, B, A		B, A
B	C, B		B
C	C	C	B, C
D	C, D	C, D	B, D
E	C, E	C, E	

$C[] = \text{best}(C[], C + B[])$

Note:

$X[]$: vector X

Y: node Y

Event 1: C receives a copy of B's vector

	New C	Old C	D
A	C, B, A	C, B, A	
B	C, B	C, B	D, B
C	C	C	D, C
D	C, D	C, D	D
E	C, E	C, E	D, E

$C[] = \text{best}(C[], C + D[])$

Event 2: C receives a copy of D's vector